```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>💰 Premium Referral System</title>
    <style>
    /* ================ PREMIUM STYLES ================ */
    * {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: 'Poppins', -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
    }

    body {
    background: linear-gradient(135deg, #6a11cb 0%, #2575fc 100%);
    min-height: 100vh;
    padding: 20px;
    color: #333;
    }

    .container {
    max-width: 500px;
    margin: 0 auto;
    }

    /* Premium Card */
    .premium-card {
    background: rgba(255, 255, 255, 0.95);
    backdrop-filter: blur(10px);
    border-radius: 24px;
    padding: 30px;
    margin-bottom: 25px;
    box-shadow: 0 20px 60px rgba(0,0,0,0.1);
    border: 1px solid rgba(255,255,255,0.2);
    }

    /* User Header */
    .user-header {
    text-align: center;
    padding-bottom: 20px;
    border-bottom: 2px solid rgba(0,0,0,0.05);
```

```
margin-bottom: 20px;
}

.user-name {
font-size: 28px;
font-weight: 700;
color: #2d3436;
margin-bottom: 8px;
letter-spacing: -0.3px;
}

.user-id {
font-size: 14px;
color: #636e72;
background: #f8f9fa;
padding: 10px 20px;
border-radius: 50px;
display: inline-block;
font-weight: 600;
margin-top: 10px;
}

/* Balance Card */
.balance-card {
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
color: white;
padding: 35px 30px;
border-radius: 24px;
text-align: center;
margin-bottom: 25px;
box-shadow: 0 15px 40px rgba(102, 126, 234, 0.3);
position: relative;
overflow: hidden;
}

.balance-card::before {
content: '💰';
position: absolute;
right: 30px;
top: 30px;
font-size: 40px;
opacity: 0.2;
}
```

```
.balance-label {
font-size: 16px;
opacity: 0.9;
margin-bottom: 15px;
letter-spacing: 1.5px;
font-weight: 600;
text-transform: uppercase;
}

.balance-amount {
font-size: 64px;
font-weight: 800;
margin-bottom: 10px;
text-shadow: 0 4px 12px rgba(0,0,0,0.2);
letter-spacing: -2px;
}

.balance-sub {
font-size: 16px;
opacity: 0.9;
font-weight: 600;
}

/* Withdraw Button */
.withdraw-btn {
background: linear-gradient(135deg, #4CAF50 0%, #45a049 100%);
color: white;
border: none;
border-radius: 20px;
padding: 25px;
width: 100%;
text-align: center;
cursor: pointer;
transition: all 0.4s cubic-bezier(0.175, 0.885, 0.32, 1.275);
box-shadow: 0 15px 35px rgba(76, 175, 80, 0.3);
margin-bottom: 25px;
font-size: 18px;
font-weight: 700;
letter-spacing: 0.5px;
position: relative;
overflow: hidden;
}

.withdraw-btn::before {
```

```
        content: '💰';
        position: absolute;

right: 30px;
        top: 50%;
        transform: translateY(-50%);
        font-size: 24px;
        opacity: 0.8;
        }

        .withdraw-btn:hover {
        transform: translateY(-5px);
        box-shadow: 0 20px 45px rgba(76, 175, 80, 0.4);
        }

        .withdraw-btn:disabled {
        background: linear-gradient(135deg, #ccc 0%, #999 100%);
        cursor: not-allowed;
        opacity: 0.7;
        transform: none !important;
        box-shadow: 0 10px 30px rgba(0,0,0,0.1);
        }

        /* Section Title */
        .section-title {
        font-size: 22px;
        font-weight: 700;
        color: #2d3436;
        margin-bottom: 25px;
        display: flex;
        align-items: center;
        gap: 12px;
        }

        .section-title span {
        background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
        color: white;
        width: 40px;
        height: 40px;
        border-radius: 12px;
        display: flex;
        align-items: center;
        justify-content: center;
        font-size: 20px;
```

```
}

/* Referral Link Container */
.referral-link-container {
background: linear-gradient(135deg, #f8f9fa 0%, #e9ecef 100%);
border-radius: 20px;
padding: 25px;
margin-bottom: 20px;
border: 2px dashed #dee2e6;
}

.referral-link-box {
background: white;
border-radius: 15px;
padding: 20px;
margin-bottom: 20px;
word-break: break-all;
font-family: 'Monaco', 'Courier New', monospace;
font-size: 15px;
color: #495057;
text-align: center;
cursor: pointer;
border: 2px solid transparent;
transition: all 0.3s;
font-weight: 600;
}

.referral-link-box:hover {
border-color: #667eea;
box-shadow: 0 5px 20px rgba(102, 126, 234, 0.1);
}

.referral-id-box {
background: #e8f4ff;
border-radius: 15px;
padding: 20px;
text-align: center;
border: 2px solid #b3d7ff;
}

.referral-id {
font-family: 'Monaco', 'Courier New', monospace;
font-size: 22px;
font-weight: 800;
```

```
color: #0066cc;
margin: 15px 0;
letter-spacing: 1px;
}

/* Copy Button */
.copy-btn {
width: 100%;
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
color: white;
border: none;
border-radius: 15px;
padding: 20px;
font-size: 18px;
font-weight: 700;
cursor: pointer;
transition: all 0.3s ease;
margin-top: 15px;
letter-spacing: 0.5px;
}

.copy-btn:hover {
transform: translateY(-3px);
box-shadow: 0 10px 25px rgba(102, 126, 234, 0.4);
}

/* Recent Referrals */
.referrals-list {
max-height: 300px;
overflow-y: auto;
margin-top: 20px;
}

.referral-item {
display: flex;
justify-content: space-between;
align-items: center;
padding: 15px;
background: #f8f9fa;
border-radius: 15px;
margin-bottom: 10px;
border-left: 4px solid #667eea;
}
```

```
        .referral-item:nth-child(even) {
        background: white;

}

        .referral-user-info {
        flex: 1;
        }

        .referral-user-name {
        font-weight: 700;
        color: #2d3436;
        margin-bottom: 5px;
        }

        .referral-user-id {
        font-size: 12px;
        color: #636e72;
        }

        .referral-amount {
        color: #28a745;
        font-weight: 900;
        font-size: 18px;
        }

        /* Messages */
        .message {
        padding: 20px;
        border-radius: 15px;
        margin: 20px 0;
        font-size: 15px;
        text-align: center;
        font-weight: 600;
        border: 2px solid transparent;
        }

        .success {
        background: linear-gradient(135deg, #d4edda 0%, #c3e6cb 100%);
        border-color: #b1dfbb;
        color: #155724;
        }

        .info {
```

```
background: linear-gradient(135deg, #d1ecf1 0%, #bee5eb 100%);
border-color: #abdde5;
color: #0c5460;
}

.error {
background: linear-gradient(135deg, #f8d7da 0%, #f5c6cb 100%);
border-color: #f1b0b7;
color: #721c24;
}

/* Footer */
.footer {
text-align: center;
color: white;
padding: 25px 0;
font-size: 14px;
opacity: 0.8;
font-weight: 600;
letter-spacing: 0.5px;
}

/* Loading */
.loading {
text-align: center;
padding: 60px 30px;
background: white;
border-radius: 25px;
}

.loading-spinner {
border: 4px solid #f3f3f3;
border-top: 4px solid #667eea;
border-radius: 50%;
width: 50px;
height: 50px;
animation: spin 1s linear infinite;
margin: 0 auto 25px;
}

@keyframes spin {
0% { transform: rotate(0deg); }
100% { transform: rotate(360deg); }
}
```

```
/* Copy Notification */
.copy-notification {
position: fixed;
top: 30px;
right: 30px;
background: linear-gradient(135deg, #28a745 0%, #20c997 100%);
color: white;
padding: 20px 30px;
border-radius: 15px;
box-shadow: 0 15px 35px rgba(0,0,0,0.2);
z-index: 1001;
animation: slideIn 0.3s cubic-bezier(0.175, 0.885, 0.32, 1.275);
display: none;
font-weight: 700;
letter-spacing: 0.5px;
}

@keyframes slideIn {
from {
        transform: translateX(100%);
        opacity: 0;
}
to {
        transform: translateX(0);
        opacity: 1;
}
}

/* Modal Styles */
.modal {
position: fixed;
top: 0;
left: 0;
width: 100%;
height: 100%;
background: rgba(0,0,0,0.8);
backdrop-filter: blur(5px);
z-index: 1000;
display: flex;
align-items: center;
justify-content: center;
padding: 20px;
animation: fadeIn 0.3s ease;
```

```
        }

        .modal-content {
        background: white;
        border-radius: 25px;
        max-width: 500px;
        width: 100%;
        max-height: 90vh;
        overflow-y: auto;
        animation: slideUp 0.4s cubic-bezier(0.175, 0.885, 0.32, 1.275);
        box-shadow: 0 25px 70px rgba(0,0,0,0.3);
        }

        .modal-header {

padding: 25px 30px;
        border-bottom: 2px solid #f8f9fa;
        display: flex;
        justify-content: space-between;
        align-items: center;
        }

        .modal-header h2 {
        font-size: 24px;
        color: #2d3436;
        font-weight: 800;
        letter-spacing: -0.5px;
        }

        .close-btn {
        background: none;
        border: none;
        font-size: 28px;
        color: #6c757d;
        cursor: pointer;
        width: 40px;
        height: 40px;
        display: flex;
        align-items: center;
        justify-content: center;
        border-radius: 10px;
        transition: all 0.3s;
        }
```

```
.close-btn:hover {
background: #f8f9fa;
color: #2d3436;
}

.modal-body {
padding: 30px;
}

@keyframes fadeIn {
from { opacity: 0; }
to { opacity: 1; }
}

@keyframes slideUp {
from {
        transform: translateY(50px);
        opacity: 0;
}
to {
        transform: translateY(0);
        opacity: 1;
}
}

/* Admin Styles (UNCHANGED) */
.admin-container {
max-width: 800px;
margin: 0 auto;
}

.admin-card {
background: white;
border-radius: 24px;
padding: 30px;
margin-bottom: 25px;
box-shadow: 0 20px 60px rgba(0,0,0,0.1);
}

.admin-header {
text-align: center;
padding: 35px;
background: linear-gradient(135deg, #2d3436 0%, #4a6491 100%);
color: white;
```

```
border-radius: 24px;
margin-bottom: 25px;
}

.admin-header h1 {
font-size: 32px;
font-weight: 800;
margin-bottom: 15px;
letter-spacing: -0.5px;
}

.admin-header p {
font-size: 16px;
opacity: 0.9;
font-weight: 600;
}

.card-title {
font-size: 24px;
font-weight: 700;
color: #2d3436;
margin-bottom: 25px;
padding-bottom: 20px;
border-bottom: 3px solid #f8f9fa;
letter-spacing: -0.5px;
}

.input-group {
margin-bottom: 25px;
}

.input-group label {
display: block;
font-size: 15px;
font-weight: 700;
color: #495057;
margin-bottom: 12px;
letter-spacing: 0.5px;
}

.input-group input,
.input-group textarea,
.input-group select {
width: 100%;
```

```
        padding: 18px;
        border: 2px solid #e9ecef;
        border-radius: 15px;
        font-size: 16px;
        transition: all 0.3s;
        font-weight: 600;
        background: #f8f9fa;
        }

        .input-group input:focus,
        .input-group textarea:focus,
        .input-group select:focus {
        outline: none;
        border-color: #667eea;
        background: white;
        box-shadow: 0 5px 20px rgba(102, 126, 234, 0.1);
        }

        .toggle-label {
        display: flex;
        align-items: center;
        margin-bottom: 25px;
        font-size: 16px;
        color: #495057;
        font-weight: 600;
        }

        .toggle-switch {
        position: relative;
        display: inline-block;
        width: 70px;

height: 34px;
        margin-right: 20px;
        }

        .toggle-switch input {
        opacity: 0;
        width: 0;
        height: 0;
        }

        .toggle-slider {
        position: absolute;
```

```
cursor: pointer;
top: 0;
left: 0;
right: 0;
bottom: 0;
background-color: #ccc;
transition: .4s;
border-radius: 34px;
}

.toggle-slider:before {
position: absolute;
content: "";
height: 26px;
width: 26px;
left: 4px;
bottom: 4px;
background-color: white;
transition: .4s;
border-radius: 50%;
}

input:checked + .toggle-slider {
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
}

input:checked + .toggle-slider:before {
transform: translateX(36px);
}

.submit-btn {
width: 100%;
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
color: white;
border: none;
border-radius: 15px;
padding: 22px;
font-size: 18px;
font-weight: 800;
cursor: pointer;
transition: all 0.3s cubic-bezier(0.175, 0.885, 0.32, 1.275);
letter-spacing: 0.5px;
}
```

```
.submit-btn:hover {
transform: translateY(-5px);
box-shadow: 0 15px 35px rgba(102, 126, 234, 0.4);
}

.withdrawals-list {
max-height: 500px;
overflow-y: auto;
}

.withdrawal-item {
background: linear-gradient(135deg, #f8f9fa 0%, #e9ecef 100%);
border-radius: 20px;
padding: 25px;
margin-bottom: 20px;
border-left: 5px solid #667eea;
}

.withdrawal-header {
display: flex;
justify-content: space-between;
align-items: center;
margin-bottom: 15px;
}

.withdrawal-user {
font-weight: 800;
color: #2d3436;
font-size: 18px;
}

.withdrawal-amount {
font-size: 28px;
font-weight: 900;
color: #28a745;
letter-spacing: -1px;
}

.withdrawal-details {
font-size: 14px;
color: #636e72;
margin-bottom: 15px;
font-weight: 600;
line-height: 1.6;
```

```
}

.withdrawal-status {
display: inline-block;
padding: 8px 20px;
border-radius: 50px;
font-size: 13px;
font-weight: 800;
margin-right: 10px;
letter-spacing: 0.5px;
}

.status-pending {
background: linear-gradient(135deg, #fff3cd 0%, #ffeaa7 100%);
color: #856404;
}

.status-paid {
background: linear-gradient(135deg, #d4edda 0%, #c3e6cb 100%);
color: #155724;
}

.status-rejected {
background: linear-gradient(135deg, #f8d7da 0%, #f5c6cb 100%);
color: #721c24;
}

.action-buttons {
display: flex;
gap: 15px;
margin-top: 20px;
}

.btn {
flex: 1;
padding: 15px;
border: none;
border-radius: 15px;
font-size: 15px;
font-weight: 800;
cursor: pointer;
transition: all 0.3s;
letter-spacing: 0.5px;
```

```
}

        .btn-success {
        background: linear-gradient(135deg, #28a745 0%, #20c997 100%);
        color: white;
        }

        .btn-danger {
        background: linear-gradient(135deg, #dc3545 0%, #c82333 100%);
        color: white;
        }

        .btn-primary {
        background: linear-gradient(135deg, #007bff 0%, #0056b3 100%);
        color: white;
        }

        .btn-warning {
        background: linear-gradient(135deg, #ffc107 0%, #e0a800 100%);
        color: #212529;
        }

        .login-container {
        max-width: 450px;
        margin: 100px auto;
        padding: 50px;
        background: white;
        border-radius: 25px;
        box-shadow: 0 25px 70px rgba(0,0,0,0.2);
        }

        .login-title {
        text-align: center;
        margin-bottom: 40px;
        color: #2d3436;
        font-size: 32px;
        font-weight: 800;
        letter-spacing: -1px;
        }

        .admin-stats-grid {
        display: grid;
        grid-template-columns: repeat(4, 1fr);
        gap: 20px;
```

```
margin-bottom: 30px;
}

.admin-stat-item {
background: linear-gradient(135deg, #f8f9fa 0%, #e9ecef 100%);
padding: 25px;
border-radius: 20px;
text-align: center;
transition: transform 0.3s;
}

.admin-stat-item:hover {
transform: translateY(-5px);
}

.admin-stat-value {
font-size: 32px;
font-weight: 900;
color: #2d3436;
margin-bottom: 10px;
letter-spacing: -1px;
}

.admin-stat-label {
font-size: 13px;
color: #636e72;
font-weight: 800;
text-transform: uppercase;
letter-spacing: 1px;
}

/* Responsive */
@media (max-width: 768px) {
.container, .admin-container {
        padding: 10px;
}

.balance-amount {
        font-size: 48px;
}

.modal-content {
        max-height: 85vh;
}
```

```
        .admin-stats-grid {
                grid-template-columns: repeat(2, 1fr);
        }

        .action-buttons {
                flex-direction: column;
        }
        }

        @media (max-width: 480px) {
        .balance-amount {
                font-size: 36px;
        }

        .modal-header {
                padding: 20px;
        }

        .modal-body {
                padding: 20px;
        }
        }
        </style>
</head>
<body>
        <div class="container" id="app">
        <!-- Content loads here -->
        </div>

        <div class="copy-notification" id="copyNotification">
        ✅ Copied to clipboard!
        </div>

        <script>
        // ================= INITIALIZE STORAGE =================
        function initializeStorage() {
        if (!localStorage.getItem('users')) {
                localStorage.setItem('users', JSON.stringify({}));
        }
        if (!localStorage.getItem('settings')) {
                const defaultSettings = {
                perRefer: 3,
                minRefer: 3,
```

```
                minWithdraw: 10,
                botUsername: 'CashofDreamsupi_bot',
                botToken: '',
                adminChatId: '',
                welcomeMessage: 'Welcome!',
                withdrawEnabled: true,
                maintenanceMode: false
                };

localStorage.setItem('settings', JSON.stringify(defaultSettings));
        }
        if (!localStorage.getItem('withdrawals')) {
                localStorage.setItem('withdrawals', JSON.stringify([]));
        }
        if (!localStorage.getItem('referrals')) {
                localStorage.setItem('referrals', JSON.stringify({}));
        }
        if (!localStorage.getItem('adminPassword')) {
                localStorage.setItem('adminPassword', '02092929');
        }
        }

        // ================= PARSE TELEGRAM DATA =================
        function parseTelegramData() {
        // Get everything BEFORE the # (query string)
        const queryString = window.location.href.split('#')[0].split('?')[1] || '';
        const urlParams = new URLSearchParams(queryString);

        // Get everything AFTER the # (fragment/hash)
        const hash = window.location.hash.substring(1);

        let userId = urlParams.get('user') urlParams.get('user_id') urlParams.get('id');
        let referralCode = urlParams.get('re') urlParams.get('ref') urlParams.get('referral') ||
urlParams.get('start');

        // Parse Telegram data from hash
        let telegramData = null;

        if (hash && hash.includes('tgWebAppData')) {
                try {
                const hashParams = new URLSearchParams(hash);
                const tgWebAppData = hashParams.get('tgWebAppData');

                if (tgWebAppData) {
```

```
        const tgParams = new URLSearchParams(tgWebAppData);
        const userStr = tgParams.get('user');

        if (userStr) {
                telegramData = JSON.parse(decodeURIComponent(userStr));
                if (telegramData.id) {
                userId = telegramData.id.toString();
                }
        }
        }
        } catch (e) {
        console.error("Error parsing Telegram data:", e);
        }
}

// Generate a user ID if none exists
if (!userId) {
        userId = 'user_' + Date.now() + '_' + Math.random().toString(36).substr(2, 9);
}

return {
        userId: userId,
        telegramData: telegramData,
        referralId: referralCode
};
}

// ================= PROCESS REFERRAL =================
async function processReferral(userData) {
const { userId, telegramData, referralId } = userData;

try {
        let users = JSON.parse(localStorage.getItem('users'));
        const settings = JSON.parse(localStorage.getItem('settings'));
        let referrals = JSON.parse(localStorage.getItem('referrals'));

        // Create user if not exists
        if (!users[userId]) {
        users[userId] = createUser(userId, telegramData);
        }

        const user = users[userId];
        user.lastActive = new Date().toISOString();
```

```
                // Process referral if code exists
                if (referralId && !user.referredBy) {
                // Find referrer by referral ID
                let referrer = null;
                let referrerId = null;

                for (const [uid, u] of Object.entries(users)) {
                if (u.websiteReferralId === referralId) {
                        referrer = u;

referrerId = uid;
                        break;
                }
                }

                if (referrer) {
                // Check self-referral
                if (userId === referrerId) {
                        return {
                        success: false,
                        message: "You cannot refer yourself"
                        };
                }

                // Check if referral already processed
                const referralKey =
${referrerId}_${userId}
;
                if (referrals[referralKey]) {
                        return {
                        success: false,
                        message: "This referral has already been processed"
                        };
                }

                // Process the referral
                referrer.refers += 1;
                referrer.balance += settings.perRefer;
                referrer.totalEarned = (referrer.totalEarned || 0) + settings.perRefer;

                if (!referrer.referredUsers) referrer.referredUsers = [];
                referrer.referredUsers.unshift({
                        id: userId,
                        date: new Date().toISOString(),
```

```
                amount: settings.perRefer
            });

            user.referredBy = referrerId;

            // Record referral
            referrals[referralKey] = {
                newUser: userId,
                referrer: referrerId,
                date: new Date().toISOString(),
                amount: settings.perRefer,
                status: 'success'
            };

            // Save data
            localStorage.setItem('users', JSON.stringify(users));
            localStorage.setItem('referrals', JSON.stringify(referrals));

            return {
                success: true,
                message:
🎉 Welcome! $${settings.perRefer} added to referrer's account.
,
                referrerId: referrerId,
                amount: settings.perRefer
            };
            } else {
            return {
                success: false,
                message: "Invalid referral code"
            };
            }
            }

            // No referral code or user already referred
            localStorage.setItem('users', JSON.stringify(users));

            return {
            success: true,
            message: user.referredBy ?
            "Welcome back!" :
            "Welcome! Share your link to earn money.",
            noReferral: true
            };
```

```
        } catch (error) {
                console.error("Error:", error);
                return {
                success: false,
                message: "System error. Please try again."
                };
        }
        }

        function createUser(id, telegramData = null) {
        return {
                id: id,
                balance: 0,
                refers: 0,
                upi: '',
                pin: null,
                banned: false,
                joined: new Date().toISOString(),

telegramData: telegramData,
                withdrawHistory: [],
                referredUsers: [],
                referredBy: null,
                websiteReferralId: generateReferralId(),
                lastActive: new Date().toISOString(),
                totalEarned: 0,
                totalWithdrawn: 0
        };
        }

        function generateReferralId() {
        const chars = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789';
        let code = '';
        for (let i = 0; i < 8; i++) {
                code += chars[Math.floor(Math.random() * chars.length)];
        }
        return code;
        }

        // ================ MAIN LOADING ================
        document.addEventListener('DOMContentLoaded', function() {
        initializeStorage();
```

```
        const urlParams = new URLSearchParams(window.location.search);
        const page = urlParams.get('page');

        if (page === 'admin') {
                handleAdminPage();
                return;
        }

        const userData = parseTelegramData();

        if (userData.userId) {
                showLoading("Setting up your account...");

                setTimeout(() => {
                processReferral(userData)
                .then(result => {
                        showUserDashboard(userData.userId, userData.telegramData, result);
                })
                .catch(error => {
                        showUserDashboard(userData.userId, userData.telegramData, {
                        success: false,
                        message: "Error processing request"
                        });
                });
                }, 1000);
        } else {
                showWelcomeScreen();
        }
        });

        function showLoading(message) {
        document.getElementById('app').innerHTML =

                <div class="premium-card loading">
                <div class="loading-spinner"></div>
                <h3 style="text-align: center; color: #2d3436; font-weight:
600;">${message}</h3>
                </div>

;
        }

        function showWelcomeScreen() {
        document.getElementById('app').innerHTML =
```

```
                <div class="premium-card">
                <div class="user-header">
                <h1 style="color: #2d3436; margin-bottom: 15px;">💰 Premium Referral System</h1>
                <p style="color: #636e72; margin-bottom: 25px;">Please open this link from Telegram to continue</p>
                <button class="copy-btn" onclick="location.href='?page=admin'" style="margin-top: 20px;">
                        🔧 Admin Panel
                </button>
                </div>
                </div>

;
        }

        // ================= SIMPLIFIED USER DASHBOARD =================
        function showUserDashboard(userId, telegramData, result) {
        const users = JSON.parse(localStorage.getItem('users'));
        const settings = JSON.parse(localStorage.getItem('settings'));

        // Ensure user exists
        if (!users[userId]) {
                users[userId] = createUser(userId, telegramData);
                localStorage.setItem('users', JSON.stringify(users));
        }

        const user = users[userId];
        user.lastActive = new Date().toISOString();
        localStorage.setItem('users', JSON.stringify(users));

        const botUsername = settings.botUsername || 'CashofDreamsupi_bot';
        const telegramReferralLink = https://t.me/${botUsername}?start=${user.websiteReferralId}
;

// Calculate withdrawal eligibility
        const canWithdraw = user.balance >= settings.minWithdraw && user.refers >= settings.minRefer;
        const missingReferrals = Math.max(0, settings.minRefer - user.refers);
        const missingBalance = Math.max(0, settings.minWithdraw - user.balance);

        // Withdraw button
```

```
        let withdrawButton = '';
        if (canWithdraw && settings.withdrawEnabled) {
            withdrawButton =

            <button class="withdraw-btn" onclick="showWithdraw('${userId}')">
            💰 WITHDRAW NOW
            </button>

;
        } else {
            let disableReason = '';
            if (!settings.withdrawEnabled) {
            disableReason = 'Withdrawals are currently disabled';
            } else if (user.balance < settings.minWithdraw && user.refers < settings.minRefer)
{
            disableReason =
Need ${missingReferrals} more referrals and $${missingBalance.toFixed(2)} more balance
;
            } else if (user.refers < settings.minRefer) {
            disableReason =
Need ${missingReferrals} more referrals
;
            } else {
            disableReason =
Need $${missingBalance.toFixed(2)} more balance
;
            }

            withdrawButton =

            <button class="withdraw-btn" disabled title="${disableReason}">
            ⏳ WITHDRAW (Requirements not met)
            </button>

;
        }

        // Recent referrals (max 10)
        let recentReferralsHTML = '';
        if (user.referredUsers && user.referredUsers.length > 0) {
            const latestReferrals = user.referredUsers.slice(0, 10);
            latestReferrals.forEach((ref, index) => {
            const refUser = users[ref.id];
            if (refUser) {
```

```
                recentReferralsHTML +=

                        <div class="referral-item">
                        <div class="referral-user-info">
                        <div class="referral-user-name">${refUser.telegramData?.first_name ||
'User'}</div>
                        <div class="referral-user-id">${ref.id.slice(0, 8)}...</div>
                        </div>
                        <div class="referral-amount">
                        +$${ref.amount}
                        </div>
                        </div>

;
                }
                });
        } else {
                recentReferralsHTML = '<div style="text-align: center; color: #636e72; padding:
20px;">No referrals yet</div>';
        }

        // SIMPLIFIED DASHBOARD - Only shows essential elements
        document.getElementById('app').innerHTML =

                <!-- User Info Card -->
                <div class="premium-card">
                <div class="user-header">
                ${result.message ?

                        <div class="${result.success ? 'success' : 'error'} message">
                        ${result.message}
                        </div>

: ''}

                <div class="user-name">
                        ${user.telegramData?.first_name || 'User'}
${user.telegramData?.last_name || ''}
                </div>

                ${user.telegramData?.username ?

                        <div style="color: #667eea; margin-bottom: 10px; font-weight: 600;">
                        @${user.telegramData.username}
```

```
</div>

: ''}

                <div class="user-id">ID: ${userId}</div>
                </div>
                </div>

                <!-- Balance Card -->
                <div class="balance-card">
                <div class="balance-label">YOUR BALANCE</div>
                <div class="balance-amount">$${user.balance.toFixed(2)}</div>
                <div class="balance-sub">${user.refers} referrals • $${(user.refers *
settings.perRefer).toFixed(2)} earned</div>
                </div>

                <!-- Withdraw Button -->
                ${withdrawButton}

                <!-- Referral Section -->
                <div class="premium-card">
                <div class="section-title">
                <span>🔗</span> Your Referral Link
                </div>

                <div class="referral-link-container">
                <div style="font-size: 15px; color: #636e72; margin-bottom: 15px; font-weight:
600;">
                        Share this link to earn <strong style="color:
#28a745;">$${settings.perRefer}</strong> for every new user
                </div>

                <div class="referral-link-box" id="referralLink" onclick="copyLinkToClipboard()"
title="Click to copy">
                        ${telegramReferralLink}
                </div>

                <div class="referral-id-box">
                        <div style="font-size: 15px; color: #636e72; margin-bottom: 5px;
font-weight: 600;">
                        Your Referral ID:
                        </div>
```

```
                        <div class="referral-id" id="referralId" onclick="copyIdToClipboard()"
style="cursor: pointer;" title="Click to copy">
                        ${user.websiteReferralId}
                        </div>
                </div>

                <button class="copy-btn" onclick="copyLinkToClipboard()">
                        📋 Copy Telegram Link
                </button>
                </div>
                </div>

                <!-- Recent Referrals -->
                <div class="premium-card">
                <div class="section-title">
                <span>👥</span> Recent Referrals
                </div>
                <div class="referrals-list">
                ${recentReferralsHTML}
                </div>
                </div>

                <div class="footer">
                Premium Referral System • ${new Date().toLocaleDateString()}
                </div>

;
        }

        // ================= WITHDRAW FUNCTIONS =================
        window.showWithdraw = function(userId) {
        const settings = JSON.parse(localStorage.getItem('settings'));
        const users = JSON.parse(localStorage.getItem('users'));
        const user = users[userId];

        // Check if withdrawals are enabled
        if (!settings.withdrawEnabled) {
                alert('Withdrawals are currently disabled by admin.');
                return;
        }

        // Check minimum referrals
        if (user.refers < settings.minRefer) {
                alert(
```

```
You need at least ${settings.minRefer} referrals to withdraw. You have: ${user.refers}
);
                return;
        }

        // Check minimum balance
        if (user.balance < settings.minWithdraw) {
                alert(
You need at least $${settings.minWithdraw} to withdraw. Your balance:
$${user.balance.toFixed(2)}
);

return;
        }

        const modal = document.createElement('div');
        modal.className = 'modal active';
        modal.innerHTML =

                <div class="modal-content">
                <div class="modal-header">
                <h2>💰 Withdraw Funds</h2>
                <button class="close-btn" onclick="this.closest('.modal').remove()">×</button>
                </div>
                <div class="modal-body">
                <div style="background: #d4edda; padding: 15px; border-radius: 10px;
margin-bottom: 20px;">
                        <div style="display: flex; justify-content: space-between; align-items:
center;">
                        <div>
                        <div style="font-weight: 700; color: #155724;">Available Balance</div>
                        <div style="font-size: 24px; font-weight: 900; color:
#28a745;">$${user.balance.toFixed(2)}</div>
                        </div>
                        <div style="font-size: 30px;">💰</div>
                        </div>
                </div>

                <div class="input-group">
                        <label>Enter your 4-digit PIN</label>
                        <input type="password" id="withdrawPin" maxlength="4"
placeholder="0000" style="text-align: center; letter-spacing: 10px;">
                        <div style="font-size: 12px; color: #6c757d; margin-top: 5px;">
```

```
                        ${user.pin ? 'Enter your existing PIN' : 'Create a new 4-digit PIN for
withdrawals'}
                        </div>
                </div>

                <button class="submit-btn" onclick="verifyWithdrawPin('${userId}')">
                        🔒 Verify & Continue
                </button>
                </div>
                </div>

;
        document.body.appendChild(modal);
        };

        window.verifyWithdrawPin = function(userId) {
        const pin = document.getElementById('withdrawPin').value;
        const users = JSON.parse(localStorage.getItem('users'));
        const user = users[userId];

        if (!pin pin.length !== 4 !/^\d+$/.test(pin)) {
                alert('PIN must be 4 digits');
                return;
        }

        if (!user.pin) {
                user.pin = pin;
                localStorage.setItem('users', JSON.stringify(users));
                showWithdrawForm(userId);
        } else {
                if (pin !== user.pin) {
                alert('Incorrect PIN');
                return;
                }
                showWithdrawForm(userId);
        }
        };

        function showWithdrawForm(userId) {
        const users = JSON.parse(localStorage.getItem('users'));
        const settings = JSON.parse(localStorage.getItem('settings'));
        const user = users[userId];

        const modal = document.querySelector('.modal.active .modal-content');
```

```
        modal.innerHTML = `
                <div class="modal-header">
                <h2>💰 Withdraw Funds</h2>
                <button class="close-btn"
onclick="document.querySelector('.modal.active').remove()">×</button>
                </div>
                <div class="modal-body">
                <div class="input-group">
                <label>UPI ID (for payment)</label>
                <input type="text" id="upiInput" placeholder="yourname@upi" value="${user.upi
|| ''}">
                </div>

                <div class="input-group">

<label>Amount to Withdraw ($)</label>
                <input type="number" id="amountInput"
                        min="${settings.minWithdraw}"
                        max="${user.balance}"
                        step="0.01"
                        value="${Math.min(user.balance, Math.max(settings.minWithdraw, 20))}"
                        style="font-size: 18px; font-weight: 700;">
                <div style="font-size: 13px; color: #6c757d; margin-top: 5px;">
                        Available: <strong>$${user.balance.toFixed(2)}</strong> |
                        Minimum: <strong>$${settings.minWithdraw}</strong>
                </div>
                </div>

                <button class="submit-btn" onclick="submitWithdrawal('${userId}')"
style="background: linear-gradient(135deg, #28a745 0%, #20c997 100%);">
                ✅ Request Withdrawal
                </button>
                </div>

;
        };

        window.submitWithdrawal = async function(userId) {
        const settings = JSON.parse(localStorage.getItem('settings'));
        const upi = document.getElementById('upiInput').value.trim();
        const amount = parseFloat(document.getElementById('amountInput').value);

        const users = JSON.parse(localStorage.getItem('users'));
        const withdrawals = JSON.parse(localStorage.getItem('withdrawals'));
```

```
	const user = users[userId];

	// Validate
	if (!upi) {
		alert('Please enter UPI ID');
		return;
	}

	if (!upi.includes('@')) {
		alert('Please enter a valid UPI ID (format: name@upi)');
		return;
	}

	if (!amount || amount < settings.minWithdraw) {
		alert(
Minimum withdrawal is $${settings.minWithdraw}
);
		return;
	}

	if (amount > user.balance) {
		alert('Insufficient balance');
		return;
	}

	if (user.refers < settings.minRefer) {
		alert(
You need at least ${settings.minRefer} referrals to withdraw
);
		return;
	}

	// Process withdrawal
	user.balance -= amount;
	user.upi = upi;
	user.lastWithdrawAttempt = new Date().toISOString();

	const withdrawal = {
		id: Date.now(),
		uid: userId,
		upi: upi,
		amount: amount,
		date: new Date().toLocaleString(),
		status: 'Pending'
```

```
        };

        user.withdrawHistory.push(withdrawal);
        withdrawals.push(withdrawal);

        localStorage.setItem('users', JSON.stringify(users));
        localStorage.setItem('withdrawals', JSON.stringify(withdrawals));

        document.querySelector('.modal.active').remove();

        // Show success message
        const successModal = document.createElement('div');
        successModal.className = 'modal active';
        successModal.innerHTML =

                <div class="modal-content">
                <div class="modal-header">
                <h2>✅ Success!</h2>
                <button class="close-btn" onclick="this.closest('.modal').remove();
location.reload();">×</button>
                </div>
                <div class="modal-body">
                <div style="text-align: center; padding: 30px 0;">
                        <div style="font-size: 60px; margin-bottom: 20px;">🎉</div>

<h3 style="color: #28a745; margin-bottom: 15px;">Withdrawal Request Submitted!</h3>
                        <div style="font-size: 24px; font-weight: 900; color: #2d3436;
margin-bottom: 10px;">
                        $${amount.toFixed(2)}
                        </div>
                        <div style="color: #636e72; margin-bottom: 20px;">
                        UPI: ${upi}<br>
                        Status: Pending
                        </div>
                        <button class="submit-btn" onclick="this.closest('.modal').remove();
location.reload();">
                        🔄 Refresh Page
                        </button>
                </div>
                </div>
                </div>

;
        document.body.appendChild(successModal);
```

```
};

// ================= ADMIN FUNCTIONS (UNCHANGED) =================
function handleAdminPage() {
const isAuthenticated = sessionStorage.getItem('adminAuthenticated') === 'true';

if (!isAuthenticated) {
        showAdminLogin();
} else {
        showAdminPanel();
}
}

function showAdminLogin() {
document.getElementById('app').innerHTML =

        <div class="login-container">
        <div class="login-title">🔐 Admin Login</div>
        <div style="margin: 40px 0;">
        <div class="input-group">
                <label>Password</label>
                <input type="password" id="adminPassword" placeholder="Enter admin
password" style="text-align: center;">
        </div>
        </div>
        <button class="submit-btn" onclick="adminLogin()">
        🔑 Login
        </button>
        <div style="text-align: center; margin-top: 25px; color: #636e72; font-size: 14px;
font-weight: 600;">
        Default password: 02092929
        </div>
        </div>

;
}

window.adminLogin = function() {
const password = document.getElementById('adminPassword').value;
const storedPassword = localStorage.getItem('adminPassword') || '02092929';

if (password === storedPassword) {
        sessionStorage.setItem('adminAuthenticated', 'true');
        showAdminPanel();
```

```
        } else {
                alert('Incorrect password! Default: 02092929');
        }
        };

        function showAdminPanel() {
        const users = JSON.parse(localStorage.getItem('users'));
        const withdrawals = JSON.parse(localStorage.getItem('withdrawals'));
        const settings = JSON.parse(localStorage.getItem('settings'));
        const referrals = JSON.parse(localStorage.getItem('referrals'));

        // Calculate statistics
        const totalUsers = Object.keys(users).length;
        const totalBalance = Object.values(users).reduce((sum, user) => sum + (user.balance ||
0), 0);
        const totalWithdrawals = withdrawals.reduce((sum, w) => sum + (w.amount || 0), 0);
        const pendingWithdrawals = withdrawals.filter(w => !w.status || w.status ===
'Pending').length;

        document.getElementById('app').innerHTML =

                <div class="admin-container">
                <!-- Admin Header -->
                <div class="admin-header">
                <h1>🔧 Admin Panel</h1>
                <p>Total Users: ${totalUsers} | Total Balance: $${totalBalance.toFixed(2)} |
Pending Withdrawals: ${pendingWithdrawals}</p>

<div style="margin-top: 25px;">
                        <button class="btn-primary" onclick="adminLogout()" style="margin-right:
10px;">Logout</button>
                        <button class="btn-success" onclick="showChangePassword()">Change
Password</button>
                </div>
                </div>

                <!-- System Stats -->
                <div class="admin-card">
                <div class="card-title">📊 System Statistics</div>
                <div class="admin-stats-grid">
                        <div class="admin-stat-item">
                        <div class="admin-stat-value">${totalUsers}</div>
                        <div class="admin-stat-label">Total Users</div>
                        </div>
```

```
                        <div class="admin-stat-item">
                        <div class="admin-stat-value">$${totalBalance.toFixed(2)}</div>
                        <div class="admin-stat-label">Total Balance</div>
                        </div>
                        <div class="admin-stat-item">
                        <div class="admin-stat-value">${Object.keys(referrals).length}</div>
                        <div class="admin-stat-label">Total Referrals</div>
                        </div>
                        <div class="admin-stat-item">
                        <div class="admin-stat-value">${withdrawals.length}</div>
                        <div class="admin-stat-label">Withdrawals</div>
                        </div>
                </div>
                </div>

                <!-- Bot Configuration -->
                <div class="admin-card">
                <div class="card-title">🤖 Bot Configuration</div>

                <div class="input-group">
                        <label>Bot Token (from @BotFather)</label>
                        <input type="text" id="botToken" value="${settings.botToken || ''}"
placeholder="1234567890:ABCdefGhIjKlMnOpQrStUvWxYz">
                </div>

                <div class="input-group">
                        <label>Bot Username (without @)</label>
                        <input type="text" id="botUsername" value="${settings.botUsername || ''}"
placeholder="yourbotusername">
                </div>

                <div class="input-group">
                        <label>Admin Chat ID (for notifications)</label>
                        <input type="text" id="adminChatId" value="${settings.adminChatId || ''}"
placeholder="123456789">
                </div>

                <button class="submit-btn" onclick="saveBotSettings()">
                        💾 Save Bot Settings
                </button>
                </div>

                <!-- Referral Settings -->
                <div class="admin-card">
```

```
                <div class="card-title">💰 Referral Settings</div>

                <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 20px;
margin-bottom: 25px;">
                        <div class="input-group">
                        <label>Bonus per Referral ($)</label>
                        <input type="number" id="perRefer" value="${settings.perRefer}"
step="0.01" min="0">
                        </div>
                        <div class="input-group">
                        <label>Minimum Referrals Required</label>
                        <input type="number" id="minRefer" value="${settings.minRefer || 3}"
min="0">

</div>
                </div>

                <div class="input-group">
                        <label>Minimum Withdrawal Amount ($)</label>
                        <input type="number" id="minWithdraw" value="${settings.minWithdraw}"
step="0.01" min="0">
                </div>

                <div class="toggle-label">
                        <label class="toggle-switch">
                        <input type="checkbox" id="withdrawEnabled"
${settings.withdrawEnabled ? 'checked' : ''}>
                        <span class="toggle-slider"></span>
                        </label>
                        <span>Enable Withdrawals</span>
                </div>

                <button class="submit-btn" onclick="saveReferralSettings()">
                        💾 Save Referral Settings
                </button>
                </div>

                <!-- Withdrawal Management -->
                <div class="admin-card">
                <div class="card-title">💸 Withdrawal Requests (${withdrawals.length})</div>
                <div class="withdrawals-list">
                        ${withdrawals.length === 0 ?
                        '<div class="info message">No withdrawal requests yet</div>' :
                        withdrawals.slice().reverse().map(w => {
```

```
                        const user = users[w.uid];
                        return

                        <div class="withdrawal-item">
                                <div class="withdrawal-header">
                                <div>
                                <div class="withdrawal-user">${user?.telegramData?.first_name ||
w.uid}</div>
                                <div style="font-size: 12px; color: #6c757d;">
                                        Refers: ${user?.refers || 0} | Balance:
$${user?.balance?.toFixed(2) || '0.00'}
                                </div>
                                </div>
                                <div class="withdrawal-amount">$${w.amount.toFixed(2)}</div>
                                </div>
                                <div class="withdrawal-details">
                                UPI: ${w.upi}<br>
                                Date: ${w.date}<br>
                                Status: <span class="withdrawal-status status-${w.status ?
w.status.toLowerCase() : 'pending'}">
                                ${w.status || 'Pending'}
                                </span>
                                </div>
                                ${(!w.status || w.status === 'Pending') ?

                                <div class="action-buttons">
                                <button class="btn-success" onclick="processWithdrawal(${w.id},
'Paid')">✅ Mark as Paid</button>
                                <button class="btn-danger" onclick="processWithdrawal(${w.id},
'Rejected')">❌ Reject</button>
                                </div>

: ''}
                        </div>

;
                        }).join('')
                        }
                </div>
                </div>

                <!-- User Management -->
                <div class="admin-card">
```

```
<div class="card-title">👥 User Management</div>

                <div class="input-group">
                        <label>Search User (by ID or Username)</label>
                        <input type="text" id="searchUser" placeholder="Enter user ID or
@username">
                </div>

                <div style="display: flex; gap: 10px; margin-bottom: 20px;">
                        <button class="btn-primary" onclick="searchAdminUser()" style="flex: 1;">
                        🔍 Search User
                        </button>
                        <button class="btn-warning" onclick="showAllUsers()" style="flex: 1;">
                        👁️ View All Users
                        </button>
                </div>

                <div id="userSearchResults">
                        <!-- Search results will appear here -->
                </div>
                </div>
                </div>

;
        }

        window.saveBotSettings = function() {
        const settings = JSON.parse(localStorage.getItem('settings'));
        settings.botToken = document.getElementById('botToken').value;
        settings.botUsername = document.getElementById('botUsername').value;
        settings.adminChatId = document.getElementById('adminChatId').value;

        localStorage.setItem('settings', JSON.stringify(settings));
        alert('Bot settings saved successfully!');
        showAdminPanel();
        };

        window.saveReferralSettings = function() {
        const settings = JSON.parse(localStorage.getItem('settings'));
        settings.perRefer = parseFloat(document.getElementById('perRefer').value) || 5;
        settings.minRefer = parseInt(document.getElementById('minRefer').value) || 3;
        settings.minWithdraw = parseFloat(document.getElementById('minWithdraw').value) ||
20;
        settings.withdrawEnabled = document.getElementById('withdrawEnabled').checked;
```

```
        localStorage.setItem('settings', JSON.stringify(settings));
        alert('Referral settings saved successfully!');
        showAdminPanel();
        };

        window.processWithdrawal = async function(withdrawalId, status) {
        const withdrawals = JSON.parse(localStorage.getItem('withdrawals'));
        const users = JSON.parse(localStorage.getItem('users'));
        const settings = JSON.parse(localStorage.getItem('settings'));

        const withdrawal = withdrawals.find(w => w.id == withdrawalId);
        if (!withdrawal) return;

        withdrawal.status = status;

        const user = users[withdrawal.uid];
        if (!user) {
                alert("User not found");
                return;
        }

        if (status === 'Rejected') {
                user.balance += withdrawal.amount;
        } else if (status === 'Paid') {
                user.totalWithdrawn = (user.totalWithdrawn || 0) + withdrawal.amount;
        }

        localStorage.setItem('withdrawals', JSON.stringify(withdrawals));
        localStorage.setItem('users', JSON.stringify(users));

        alert(
Withdrawal marked as ${status}`);
        showAdminPanel();
        };

        window.searchAdminUser = function() {
        const searchTerm = document.getElementById('searchUser').value.toLowerCase();
        const users = JSON.parse(localStorage.getItem('users'));

        let results = [];
        for (const [userId, user] of Object.entries(users)) {

const username = user.telegramData?.username?.toLowerCase() || '';
```

```
                const firstName = user.telegramData?.first_name?.toLowerCase() || '';

                if (userId.includes(searchTerm) ||
                username.includes(searchTerm) ||
                firstName.includes(searchTerm) ||
                user.websiteReferralId?.includes(searchTerm)) {
                results.push({ userId, user });
                }
        }

        let html = '';
        if (results.length === 0) {
                html = '<div class="info message">No users found</div>';
        } else {
                html =
<h4 style="margin-bottom: 15px;">Found ${results.length} users:</h4>
;
                results.forEach(({ userId, user }) => {
                html +=

                <div style="padding: 15px; background: #f8f9fa; border-radius: 15px;
margin-bottom: 10px;">
                        <div style="display: flex; justify-content: space-between; align-items:
center; margin-bottom: 10px;">
                        <div>
                        <strong>${user.telegramData?.first_name || 'Unknown'}</strong>
                        <div style="font-size: 12px; color: #636e72;">
                                @${user.telegramData?.username || 'no-username'} • ID:
${userId}
                        </div>
                        </div>
                        <div style="font-size: 20px; font-weight: 900; color: #28a745;">
                        $${user.balance.toFixed(2)}
                        </div>
                        </div>
                        <div style="display: grid; grid-template-columns: repeat(3, 1fr); gap: 10px;
margin-top: 15px;">
                        <button class="btn-primary" onclick="modifyUserBalance('${userId}')"
style="padding: 8px; font-size: 12px;">
                        💰 Edit Balance
                        </button>
                        <button class="btn-warning" onclick="viewUserDetails('${userId}')"
style="padding: 8px; font-size: 12px;">
                        👁️ View Details
```

```
                        </button>
                        <button class="${user.banned ? 'btn-success' : 'btn-danger'}"
onclick="toggleBanUser('${userId}', ${!user.banned})" style="padding: 8px; font-size: 12px;">
                        ${user.banned ? 'Unban' : 'Ban'}
                        </button>
                        </div>
                </div>

;
                });
        }

        document.getElementById('userSearchResults').innerHTML = html;
        };

        window.modifyUserBalance = function(userId) {
        const users = JSON.parse(localStorage.getItem('users'));
        const user = users[userId];

        if (!user) {
                alert("User not found");
                return;
        }

        const action = prompt(
Modify balance for ${user.telegramData?.first_name || userId}\n\nCurrent balance:
$${user.balance.toFixed(2)}\n\nEnter:\n+amount to add\n-amount to deduct\namount to set
);
        if (action === null) return;

        const amount = parseFloat(action);
        if (isNaN(amount)) {
                alert("Invalid amount");
                return;
        }

        const reason = prompt("Reason for modification:");
        if (reason === null) return;

        if (action.startsWith('+')) {
                user.balance += amount;
        } else if (action.startsWith('-')) {

user.balance -= amount;
```

```
                if (user.balance < 0) user.balance = 0;
        } else {
                user.balance = amount;
        }

        localStorage.setItem('users', JSON.stringify(users));
        alert(
Balance updated!\nNew balance: $${user.balance.toFixed(2)}
);
        showAdminPanel();
        };

        window.viewUserDetails = function(userId) {
        const users = JSON.parse(localStorage.getItem('users'));
        const user = users[userId];
        const settings = JSON.parse(localStorage.getItem('settings'));

        if (!user) {
                alert("User not found");
                return;
        }

        let details =
👤 User Details\n
;
        details +=
═══════════════════════\n\n
;
        details +=
Name: ${user.telegramData?.first_name || 'Unknown'} ${user.telegramData?.last_name || ''}\n
;
        details +=
Username: @${user.telegramData?.username || 'N/A'}\n
;
        details +=
User ID: ${userId}\n
;
        details +=
Referral ID: ${user.websiteReferralId}\n\n
;
        details +=
Balance: $${user.balance.toFixed(2)}\n
;
        details +=
```

```
Referrals: ${user.refers}\n
;
        details +=
Earned: $${(user.refers * settings.perRefer).toFixed(2)}\n
;
        details +=
Withdrawn: $${user.totalWithdrawn?.toFixed(2) || '0.00'}\n\n
;
        details +=
Joined: ${new Date(user.joined).toLocaleString()}\n
;
        details +=
Last Active: ${new Date(user.lastActive).toLocaleString()}\n
;
        details +=
Status: ${user.banned ? '❌ BANNED' : '✅ ACTIVE'}\n
;

        if (user.referredBy) {
                details +=
Referred by: ${user.referredBy}\n
;
        }

        alert(details);
        };

        window.toggleBanUser = function(userId, ban) {
        const users = JSON.parse(localStorage.getItem('users'));
        const user = users[userId];

        if (!user) {
                alert("User not found");
                return;
        }

        if (ban) {
                const reason = prompt("Enter ban reason:");
                if (!reason) return;

                user.banned = true;
                user.banReason = reason;
                user.banDate = new Date().toISOString();
                alert(
```

```
User ${userId} has been banned.
);
        } else {
                user.banned = false;
                user.banReason = '';
                user.banDate = null;
                alert(
User ${userId} has been unbanned.
);
        }

        localStorage.setItem('users', JSON.stringify(users));
        showAdminPanel();
        };

        window.showAllUsers = function() {
        const users = JSON.parse(localStorage.getItem('users'));
        const sortedUsers = Object.entries(users)
                .sort((a, b) => b[1].balance - a[1].balance)
                .slice(0, 20);

        let list =
👥 Top 20 Users by Balance\n
;
        list +=
═══════════════════════════════\n\n
;

        sortedUsers.forEach(([userId, user], index) => {
                list +=
${index + 1}. ${user.telegramData?.first_name || 'Unknown'} (@${user.telegramData?.username
|| 'N/A'})\n
;
                list +=
  Balance: $${user.balance.toFixed(2)} | Refers: ${user.refers}\n
;
                list +=
  ID: ${userId}\n\n
;
        });

        alert(list);
        };
```

```
        window.showChangePassword = function() {
        const modal = document.createElement('div');
        modal.className = 'modal active';
        modal.innerHTML = `

<div class="modal-content">
                <div class="modal-header">
                <h2>🔐 Change Admin Password</h2>
                <button class="close-btn" onclick="this.closest('.modal').remove()">×</button>
                </div>
                <div class="modal-body">
                <div class="input-group">
                        <label>Current Password</label>
                        <input type="password" id="currentPassword" placeholder="Current
password">
                </div>
                <div class="input-group">
                        <label>New Password</label>
                        <input type="password" id="newPassword" placeholder="New password
(min 4 characters)">
                </div>
                <div class="input-group">
                        <label>Confirm New Password</label>
                        <input type="password" id="confirmPassword" placeholder="Confirm new
password">
                </div>
                <button class="submit-btn" onclick="changeAdminPassword()">
                        🔑 Change Password
                </button>
                </div>
                </div>
        `;
        document.body.appendChild(modal);
        };

        function changeAdminPassword() {
        const currentPassword = document.getElementById('currentPassword').value;
        const newPassword = document.getElementById('newPassword').value;
        const confirmPassword = document.getElementById('confirmPassword').value;

        const storedPassword = localStorage.getItem('adminPassword') || '02092929';

        if (!currentPassword || currentPassword !== storedPassword) {
                alert('Current password is incorrect');
```

```
        return;
}

if (!newPassword || newPassword.length < 4) {
        alert('New password must be at least 4 characters');
        return;
}

if (newPassword !== confirmPassword) {
        alert('New passwords do not match');
        return;
}

localStorage.setItem('adminPassword', newPassword);
document.querySelector('.modal.active').remove();
alert('Password changed successfully! Logging out...');

setTimeout(() => {
        sessionStorage.removeItem('adminAuthenticated');
        location.href = '?page=admin';
}, 1500);
};

window.adminLogout = function() {
sessionStorage.removeItem('adminAuthenticated');
location.href = '?page=admin';
};

// ================= UTILITY FUNCTIONS =================
window.copyLinkToClipboard = function() {
const link = document.getElementById('referralLink').textContent;
navigator.clipboard.writeText(link).then(() => {
        const notification = document.getElementById('copyNotification');
        notification.style.display = 'block';
        setTimeout(() => notification.style.display = 'none', 2000);
});
};

window.copyIdToClipboard = function() {
const id = document.getElementById('referralId').textContent;
navigator.clipboard.writeText(id).then(() => {
        const notification = document.getElementById('copyNotification');
        notification.style.display = 'block';
        setTimeout(() => notification.style.display = 'none', 2000);
```

```
        });
        };
        </script>
</body>
</html>
```